

Visualizing Data

Distributions, proportions, and associations



Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Learning objectives and route through the class

By the end of this class, you should be able to choose a chart for a distribution, a proportion, or an association; explain its axes and statistical conventions; compare its message with another view of the same data; and reproduce a labelled example in R or Python. The two languages are alternatives. Their paired examples make differences in defaults visible rather than requiring both implementations.

Class A introduced visual design, chart choice, and amounts by category. This class builds on those ideas and on the cleaning and transformation work from Week 2. We first examine one-variable distributions, then compare groups, show parts of a whole, and finish with two-variable and multiple-variable associations. The practice at the end supports Lab 03.

A chart is an analytical choice. Bins, bandwidths, units, denominators, groupings, and axis limits can change what a reader notices. A good caption makes the relevant choices visible and limits the claim to what the data support.

1 Recap and chart vocabulary

1.1 Make earlier observations explicit

Class A used lagged variables to represent earlier measurements as new columns. For an ordered series x_t , a lag of one is x_{t-1} and a lag of two is x_{t-2} . A scatter plot of x_t against x_{t-1} can reveal an association between successive observations without fitting a prediction model.

Sort observations by time first. If rows come from several sites or people, construct each history within its own group. A one-row shift is a one-step lag; it is not necessarily a one-day lag when the sampling intervals are irregular. Initial lags are undefined because the relevant earlier observations are absent. They are not zero, and creating a lag does not justify filling them.

Time	x_t	x_{t-1}	x_{t-2}
1	5.1	NA	NA
2	4.8	5.1	NA
3	5.4	4.8	5.1
4	5.0	5.4	4.8

A lag of one moves a value into the next row. Initial lags are undefined; they are not zero.

Figure 1. A four-observation series with two lagged columns. Values move to later rows while their original column remains available.

The same idea can describe spatial context by adding a neighbour's measurement, but the neighbour definition and distance convention must be justified. Neither lagged nor neighbouring values turn dependence into independent observations.

1.2 Organize charts by the question

Table 1. Chart families organized by their analytical question. Stacked area is an optional extension for changing composition over time.

Question	Useful chart families
How much in each category?	Bars, dots, heatmaps (Class A)
How are values distributed?	Histograms, densities, boxes, violins, ECDFs, Q-Q plots
How is a whole divided?	Bars, pies, stacked bars; stacked area for change over time
How do two variables relate?	Scatter plots
How do several variables relate?	Pair matrices, correlograms, parallel coordinates

The chart taxonomy in Table 1 offers candidates, not a one-to-one rule. A histogram and an ECDF both describe distributions, but they make different comparisons easy. A bar chart and a pie both show proportions, but only the bar chart places every part on a common baseline.

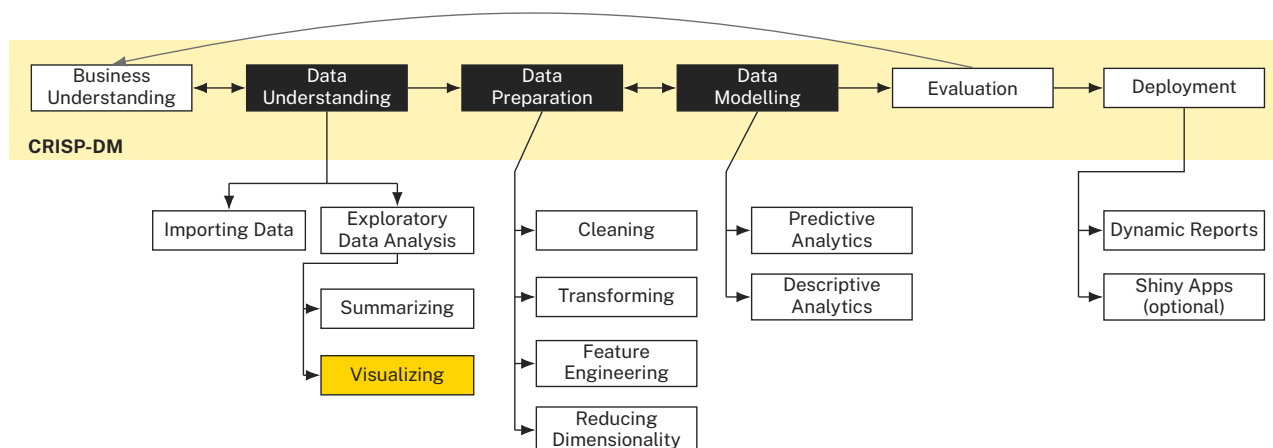


Figure 2. The course workflow, with visualizing highlighted inside exploratory data analysis. These are connected analytical activities, not a requirement to complete every future method now.

1.3 A plotting grammar separates decisions

A plotting grammar distinguishes the data, the mapping of variables to visual properties, the geometry of the marks, and the scales, labels, and theme. In ggplot2, aes maps a variable, a geom chooses marks, and + combines layers [7]. A fixed colour outside aes applies a constant style; a colour mapped inside aes encodes a variable and usually creates a legend.

Grammar-based plotting is also available in Python, including plotnine and seaborn's objects interface. The examples here use matplotlib and seaborn. A matplotlib axes object receives plotting and labelling calls, so labels must be applied to the same axes that contains the data. The distinction is one of API organization, not a limit on the charts that a language can produce.

2 Example data and a reproducible plotting session

Iris contains 150 flowers, with 50 observations for each of setosa, versicolor, and virginica. The four numeric columns are sepal length, sepal width, petal length, and petal width, all in centimetres [3, 5]. The R dataset and scikit-learn's built-in copy agree on all 600 numeric entries. No network download is needed. These data illustrate plotting choices; they do not establish how every iris population is distributed.

Run the setup once, then run the examples below in order. Choose R or Python. In an interactive session, print an R plot object, such as `print(p_histogram)`, or call `plt.show()` in Python to see it. Real course data still require the Week 2 checks for types, units, missing values, and reporting limits before they are plotted.

R

```
library(ggplot2)
library(GGally)
d <- iris
species <- levels(d$Species)
colours <- c("#242424", "#A88800", "#6E6E6E")
numeric_names <- names(d)[1:4]
```

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
from sklearn.datasets import load_iris
raw = load_iris(as_frame=True)
d = raw.data.copy()
d.columns = ["Sepal.Length", "Sepal.Width",
             "Petal.Length", "Petal.Width"]
d["Species"] = pd.Categorical.from_codes(
    raw.target, raw.target_names)
species = list(raw.target_names)
colours = ["#242424", "#A88800", "#6E6E6E"]
markers = ["o", "^", "s"]
numeric_names = list(d.columns[:4])
```

The four-row lagged example can now be reproduced using the available setup.

R

```
series <- data.frame(x = c(5.1, 4.8, 5.4, 5.0))
series$lag1 <- c(NA, head(series$x, -1))
series$lag2 <- c(NA, NA, head(series$x, -2))
print(series)
```

Python

```
series = pd.DataFrame({"x": [5.1, 4.8, 5.4, 5.0]})
series["lag1"] = series["x"].shift(1)
series["lag2"] = series["x"].shift(2)
print(series)
```

R

```
p_grammar <- ggplot(d, aes(Sepal.Length)) +
  geom_histogram(
    breaks = seq(4.25, 8.25, 0.4),
    closed = "left") +
  labs(x = "Sepal length (cm)", y = "Count") +
  theme_minimal()
```

Python

```
fig, ax = plt.subplots()
ax.hist(d["Sepal.Length"],
        bins=np.linspace(4.25, 8.25, 11))
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
```

3 Visualizing one-variable distributions

3.1 Box plots separate quartiles, fences, and whiskers

A sample five-number summary contains the minimum, first quartile Q_1 , median, third quartile Q_3 , and maximum. The interquartile range is $IQR = Q_3 - Q_1$. A conventional box plot draws the box from Q_1 to Q_3 and a line at the median. Its fences are $Q_1 - 1.5 IQR$ and $Q_3 + 1.5 IQR$; the whiskers reach the most

extreme observed values inside those fences. A point outside the whiskers is a flag for investigation, not evidence that a measurement is invalid.

For Iris petal length, the sample summary is (1.0, 1.6, 4.35, 5.1, 6.9) cm. There are no points outside the fences, so the whiskers happen to coincide with the minimum and maximum. The interval between Q_1 and the median is much wider than the interval below Q_1 . That describes spacing; it does not prove how many modes or populations produced the data. A histogram or density plot adds shape.

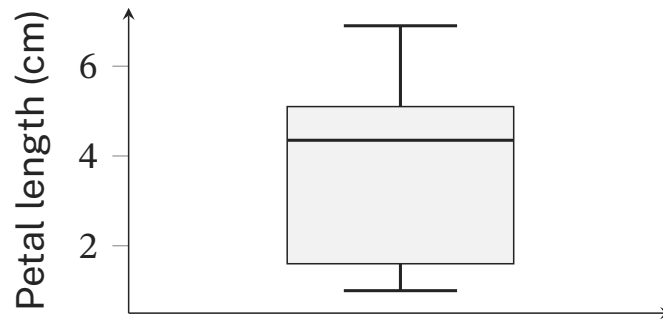


Figure 3. Petal-length box plot using linearly interpolated quartiles and the 1.5-IQR whisker rule. All plotted values are in centimetres.

R

```
x <- d$Petal.Length
print(quantile(x, c(0, .25, .5, .75, 1)))
p_box <- ggplot(d, aes(y = Petal.Length)) +
  geom_boxplot(coef = 1.5) +
  labs(y = "Petal length (cm)")
```

Python

```
x = d["Petal.Length"]
print(x.quantile([0, .25, .5, .75, 1]))
fig, ax = plt.subplots()
sns.boxplot(y=x, whis=1.5, ax=ax)
ax.set_ylabel("Petal length (cm)")
```

The examples use ggplot2's type-7 quantiles and pandas' linear-interpolation quantiles. Base R's boxplot uses Tukey hinges, which can differ from these quartiles on some samples. Record the convention when a small difference in a box edge or flagged point matters. Do not call the whisker endpoints the sample minimum and maximum unless they actually coincide.

3.2 Histograms make binning explicit

A histogram divides a numeric axis into intervals and counts observations in each one. Unlike named categories in a bar chart, these bins are selected by the analyst. For equal-width bins, height can show count. With unequal widths, a count histogram needs height equal to count divided by width, so that bar area represents count. A probability-density histogram divides that quantity by the total count as well, making the total area one.

The displayed Iris comparison uses the same interval 4.25–8.25 cm with either 30 or 10 equal-width bins. Their widths are about 0.133 cm and 0.4 cm. Measurements are recorded to 0.1 cm, so grids close to that resolution can turn the recording pattern into visible spikes. Moving an edge or changing the width can change the apparent shape without changing any observation. Neither view is the unique distribution of the flowers.

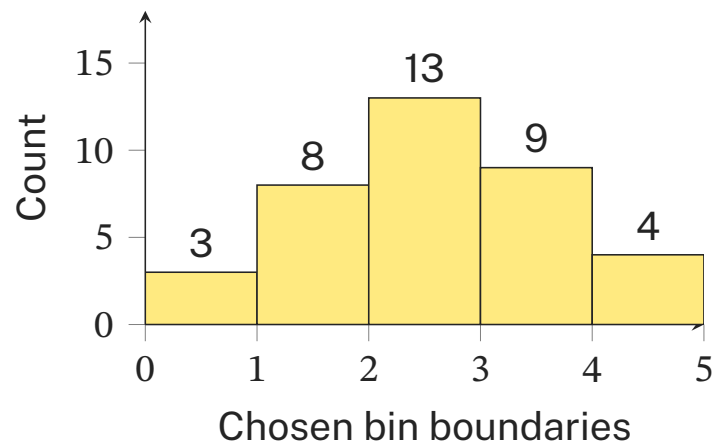


Figure 4. Five equal-width bins with counts 3, 8, 13, 9, and 4. Bar heights are calculated directly from their labelled counts.

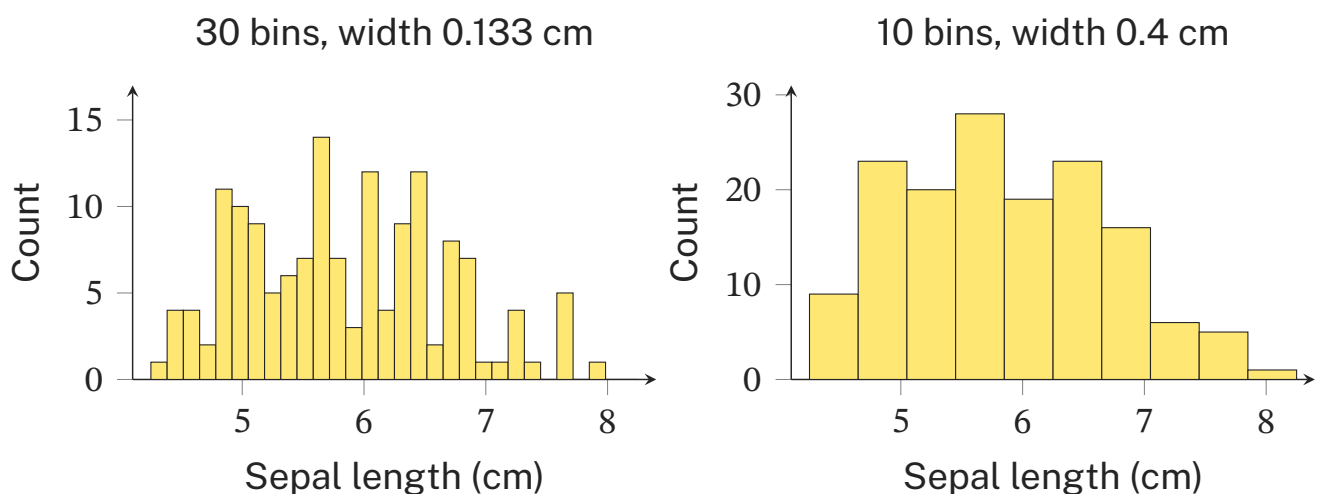


Figure 5. The same 150 sepal lengths under two explicit grids. The 10-bin view has counts 9, 23, 20, 28, 19, 23, 16, 6, 5, and 1, which sum to 150.

R

```
edges <- seq(4.25, 8.25, by = 0.4)
bins <- cut(d$Sepal.Length, edges,
  right = FALSE, include.lowest = TRUE)
print(table(bins))
p_histogram <- ggplot(d, aes(Sepal.Length)) +
  geom_histogram(
    breaks = edges, closed = "left") +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```
edges = np.linspace(4.25, 8.25, 11)
counts, _ = np.histogram(
  d["Sepal.Length"], bins=edges)
print(counts)
fig, ax = plt.subplots()
ax.hist(d["Sepal.Length"], bins=edges)
ax.set(xlabel="Sepal length (cm)",
  ylabel="Count")
```

The edges lie between the observed tenth-centimetre values, avoiding ambiguous floating-point comparisons at those particular boundaries. Both examples use left-closed intervals; NumPy includes the final right edge in its last bin. No observation in this dataset lies on that final edge. For another dataset, state and test endpoint membership directly. Equal bin counts or equal widths alone do not ensure equal histograms; the origin, complete edge vector, and closure rule also matter.

Library defaults are useful starting points rather than shared conventions. ggplot2 requests 30 bins if neither width nor breaks is supplied; base R's `hist` starts from Sturges' rule and chooses convenient boundaries; matplotlib's default is 10 bins. NumPy's `auto`, also used by seaborn histograms, selects the narrower of the Freedman–Diaconis and Sturges widths. For these sepal lengths, NumPy uses 9

bins under auto and 8 under fd. Base R can return a different number of convenient bins even when an integer request is supplied. Explicit edges make a comparison reviewable.

3.3 Nearly equal summaries need not imply equal shape

Figure 6 uses three datasets from the boxplot examples associated with Matejka and Fitzmaurice [4]. Each contains 2,484 observations and has a five-number summary close to $(-9.77, -2.69, 0, 2.68, 9.76)$. The summaries are nearly equal, not exactly equal under the linear-interpolation convention used here. Equal-width histograms nevertheless reveal very different concentrations of values.

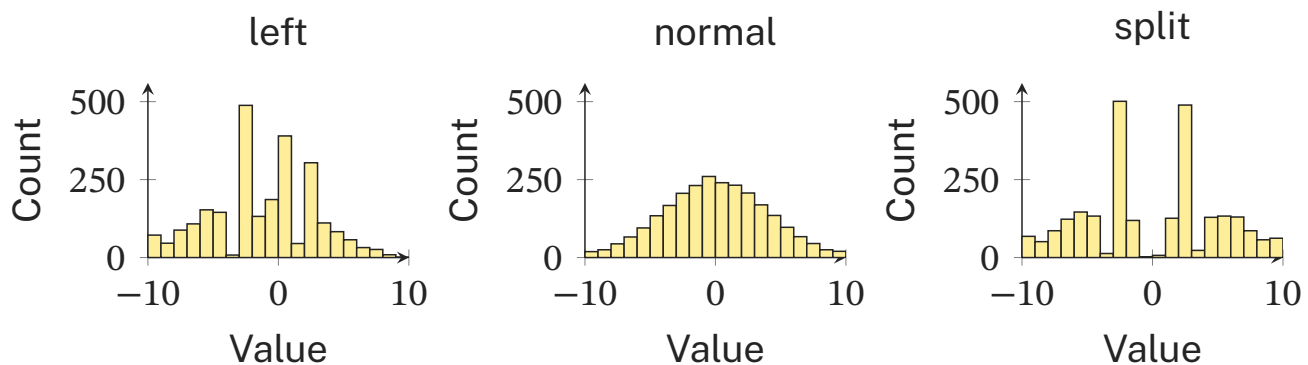


Figure 6. Source datasets named left, normal, and split, drawn from the actual observations with one shared histogram grid. The names identify the source sets; they are not diagnoses inferred from a box plot.

The observations are available in the `datsauRus` box-plot data, distributed with the package's MIT licence. These figures are computed from those observations. This comparison preserves the general lesson from Class A's scatter-plot examples. Check a view of the observations alongside a compact summary.

3.4 Kernel density estimates average smooth contributions

A kernel density estimate replaces each observation by a smooth kernel (here a bell-shaped Gaussian curve) and averages those contributions. With a kernel K whose integral is one,

$$\hat{f}_h(t) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{t - x_i}{h}\right).$$

Here n is the number of observations, x_i is observation i , and t is the location where we evaluate the curve. K is the standard-normal density shape. The bandwidth h is positive and has the same unit as the measurement. Each contribution integrates to $1/n$, so the whole curve integrates to one. For widths measured in centimetres, the vertical density has unit cm^{-1} . A density value is not the probability of one exact width; probabilities correspond to areas over intervals.

Small bandwidths preserve more local variation, including sampling noise. Large bandwidths smooth more strongly and may merge distinct features. The Iris petal-width curve in Figure 8 uses the fixed bandwidth 0.25 cm and shows a low-width concentration and a broader higher-width region. Gaussian kernels also assign some density to negative widths, which are physically impossible. This boundary artefact is a limitation of the unconstrained estimator, not a new type of flower.

In the R expression, `sapply` evaluates the calculation at each grid point and `dnorm` returns one Gaussian contribution per observation. In Python, `grid[:, None]` creates a column of grid points. Combining it with the observation vector produces a table with one row per grid point and one column per observation; `mean(axis=1)` averages each row. The formula and the code therefore perform the same averaging operation.

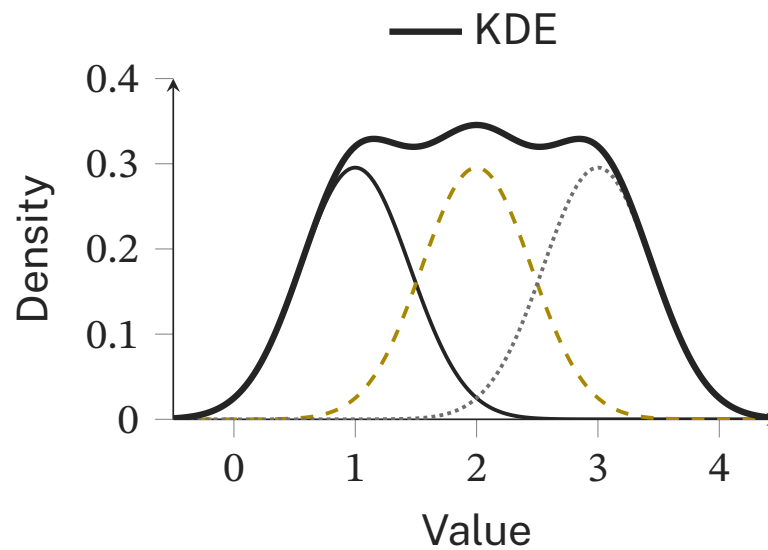


Figure 7. Three Gaussian contributions with observations 1, 2, and 3 and bandwidth 0.45. Each thinner curve has area one third; their sum is the thicker density curve with total area one.

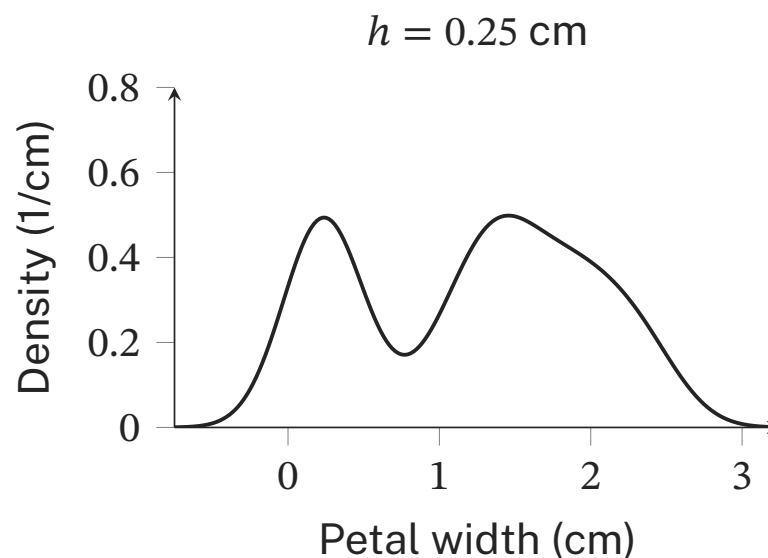


Figure 8. Petal-width Gaussian KDE with bandwidth 0.25 cm and a common evaluation grid. The curve is normalized over the full real line; the figure displays the stated finite range.

R

```
h <- 0.25 # bandwidth in cm
x <- d$Petal.Width
grid <- seq(-0.75, 3.25, length.out = 201)
f <- sapply(grid, function(t) {
  mean(dnorm(t, x, h))
})
curve <- data.frame(width = grid, density = f)
p_density <- ggplot(curve,
  aes(width, density)) + geom_line() +
  labs(x = "Petal width (cm)",
    y = "Density (1/cm)")
```

Python

```
h = 0.25 # bandwidth in cm
x = d["Petal.Width"].to_numpy()
grid = np.linspace(-0.75, 3.25, 201)
kernels = norm.pdf(grid[:, None],
  loc=x, scale=h)
f = kernels.mean(axis=1)
fig, ax = plt.subplots()
ax.plot(grid, f)
ax.set(xlabel="Petal width (cm)",
  ylabel="Density (1/cm)")
```

R's default `nrd0` bandwidth is $0.9 \min(s, \text{IQR}/1.34) n^{-1/5}$; SciPy's Scott rule uses $sn^{-1/5}$ for one-dimensional data. For these measurements the bandwidths are about 0.251834 cm and 0.279816 cm. Both estimates remain bimodal here. The formulas, not the language names, explain the difference. Some APIs accept a bandwidth in measurement units, while SciPy's `bw_method` accepts a factor that multiplies the sample SD. A fixed physical bandwidth therefore corresponds to the factor h/s for a nonconstant group.

4 Comparing distributions between groups

4.1 Counts, densities, and summaries answer different questions

All three Iris species have 50 records, but the same group-comparison methods also apply when group sizes differ. A stacked histogram adds the groups' counts within each bin. Its total height answers how many observations lie in that interval. Only the bottom group begins at the common zero baseline, so comparing an interior group's shape requires following changing segment boundaries. A stack is useful for contributions to a total; it is less direct for comparing within-group distribution shapes.

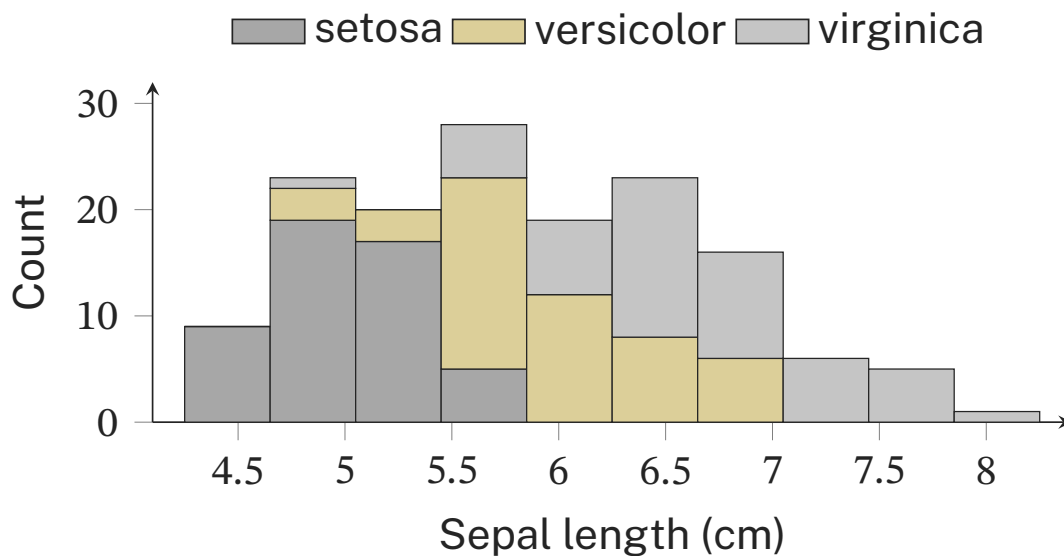


Figure 9. Species counts stacked using the same 0.4-cm bins. Stack order follows the explicit species order in both examples.

R

```
fills <- c("#A7A7A7", "#DCCF99", "#C5C5C5")
styles <- c("solid", "dashed", "dotted")
p_group_hist <- ggplot(d,
  aes(Sepal.Length, fill = Species)) +
  geom_histogram(
    breaks = edges, closed = "left",
    position = position_stack(reverse = TRUE)) +
  scale_fill_manual(values = fills) +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```
fills = ["#A7A7A7", "#DCCF99", "#C5C5C5"]
styles = ["-", "--", ":"]
fig, ax = plt.subplots()
groups = [
    d.loc[d["Species"] == s, "Sepal.Length"]
    for s in species]
ax.hist(groups, bins=edges, stacked=True,
        color=fills, label=species)
ax.set(xlabel="Sepal length (cm)",
        ylabel="Count")
ax.legend()
```

A separate density for each species answers a different question. Normalizing every curve to area one compares shape independently of group size; it does not show which group contains more records. Report group counts as well. The shared 0.20-cm bandwidth in Figure 10 keeps the amount of smoothing comparable. Setosa has shorter sepal lengths overall, but the group ranges and densities overlap.

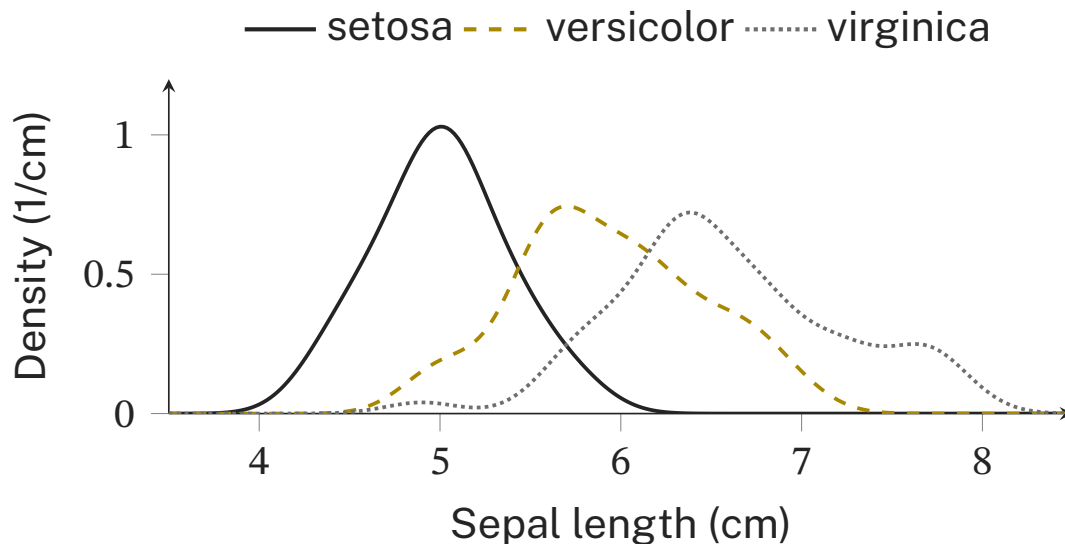


Figure 10. Gaussian group densities, each normalized to area one before display, with a shared bandwidth of 0.20 cm. Colour and line pattern identify species.

R

```
h_group <- 0.20 # common bandwidth, cm
p_group_density <- ggplot(d,
  aes(Sepal.Length, colour = Species,
      linetype = Species)) +
  geom_density(bw = h_group) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Sepal length (cm)",
        y = "Density (1/cm)")
```

Python

```
h_group = 0.20 # common bandwidth, cm
fig, ax = plt.subplots()
for values, s, c, line in zip(
    groups, species, colours, styles):
    bandwidth = h_group / values.std(ddof=1)
    sns.kdeplot(
        values, bw_method=bandwidth,
        common_norm=False, color=c,
        linestyle=line, label=s, ax=ax)
ax.set(xlabel="Sepal length (cm)",
        ylabel="Density (1/cm)")
ax.legend()
```

Seaborn's grouped kdeplot normally uses `common_norm=True`, which weights each conditional curve by its group's share of the observations. For three equal groups, that makes each curve's area one third. The examples plot and normalize groups separately to match the within-group question. R and Python use slightly different finite evaluation grids and numerical KDE algorithms, so coordinates can

differ slightly even when the estimator and bandwidth agree. The published curve bodies evaluate the stated Gaussian formula directly on a common grid.

4.2 Box plots compare a compact set of summaries

The grouped box plots emphasize location, spread, and flagged points. *Virginica*'s first quartile is 6.225 cm under linear interpolation. Its 4.9-cm observation lies beyond the lower fence; the lower whisker is 5.6 cm. The specimen might be unusual or misrecorded, but the box plot cannot decide which explanation is correct. Keep the Week 2 distinction between flagging an observation and changing it.

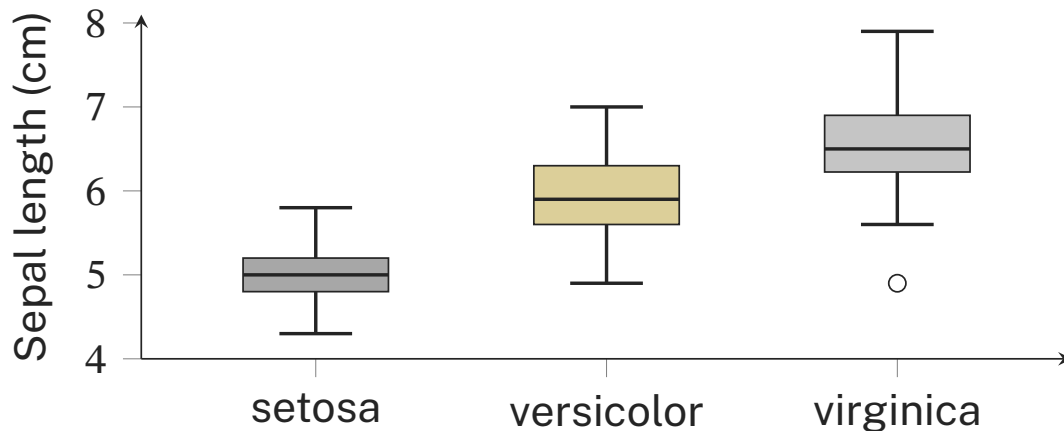


Figure 11. Sepal-length box plots using one quartile and whisker convention for all species. The low *virginica* point remains visible.

Table 2. Sepal-length box-plot summaries in centimetres. Whisker endpoints can differ from sample extrema.

Species	Lower whisker	Q ₁	Median	Q ₃	Upper whisker
setosa	4.300	4.800	5.000	5.200	5.800
versicolor	4.900	5.600	5.900	6.300	7.000
virginica	5.600	6.225	6.500	6.900	7.900

R

```
p_group_box <- ggplot(d,
  aes(Species, Sepal.Length, fill = Species)) +
  geom_boxplot(coef = 1.5) +
  scale_fill_manual(values = fills) +
  labs(x = "Species", y = "Sepal length (cm)")
```

Python

```
fig, ax = plt.subplots()
sns.boxplot(
  data=d, x="Species", y="Sepal.Length",
  hue="Species", order=species, whis=1.5,
  palette=fills, saturation=1,
  legend=False, dodge=False, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

4.3 Violins add estimated shape

A violin mirrors a density curve around its category position. The two sides are identical by construction. Skewness is read along the measurement axis, not as a left-right imbalance. Width at a given measurement indicates relative estimated density, subject to the chosen scaling convention.

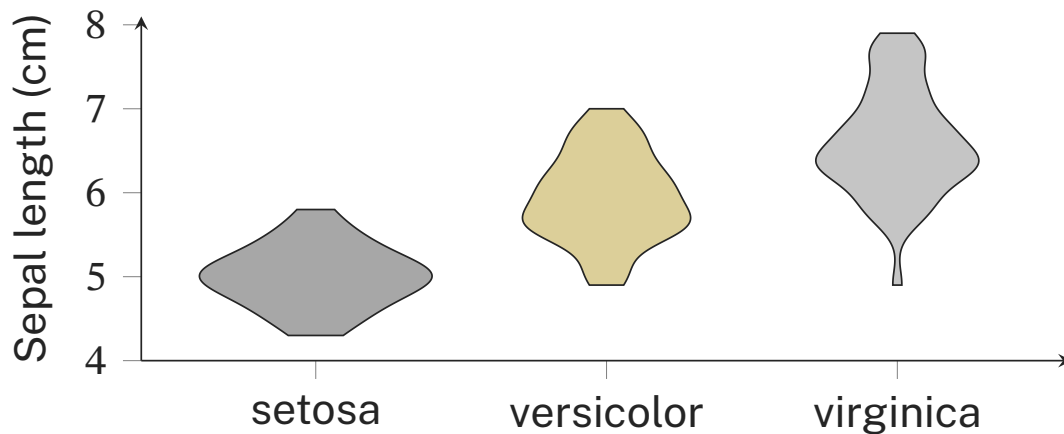


Figure 12. Sepal-length violins with Gaussian bandwidth 0.20 cm, equal-area scaling before trimming, and tails restricted to each species' observed range. Visible areas can differ after trimming.

R

```
p_violin <- ggplot(d,
  aes(Species, Sepal.Length, fill = Species)) +
  geom_violin(bw = h_group, trim = TRUE,
    scale = "area") +
  scale_fill_manual(values = fills) +
  labs(x = "Species", y = "Sepal length (cm)")
```

Python

```
fig, ax = plt.subplots()
def bandwidth(kde):
    return h_group / kde.dataset.std(ddof=1)
sns.violinplot(
  data=d, x="Species", y="Sepal.Length",
  hue="Species", order=species,
  palette=fills, saturation=1,
  legend=False, dodge=False,
  bw_method=bandwidth, cut=0,
  density_norm="area", common_norm=True,
  inner=None, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

The examples state the bandwidth, trimming, and area convention. The Python callback receives each group's density estimator and returns the factor h/s from that group's stored measurements; it fixes the physical bandwidth while allowing the groups to have different standard deviations. Seaborn's current default is `density_norm="area"`, not equal maximum width; its default `cut=2` extends tails beyond the observed range. `ggplot2` normally trims its violins. Setting a matching trimming rule alone does not match the two plots if their bandwidths or group scaling differ. The shown examples omit an internal box so the density boundary is easier to see. For a very small group, show the observations directly and treat an estimated violin shape cautiously.

Figure 13 returns to all five source boxplot datasets. The narrow internal boxes are almost indistinguishable, while the density shapes differ. The bandwidth is 0.6 in the dataset's value units. Its selection is part of the illustration and is not a universal rule.

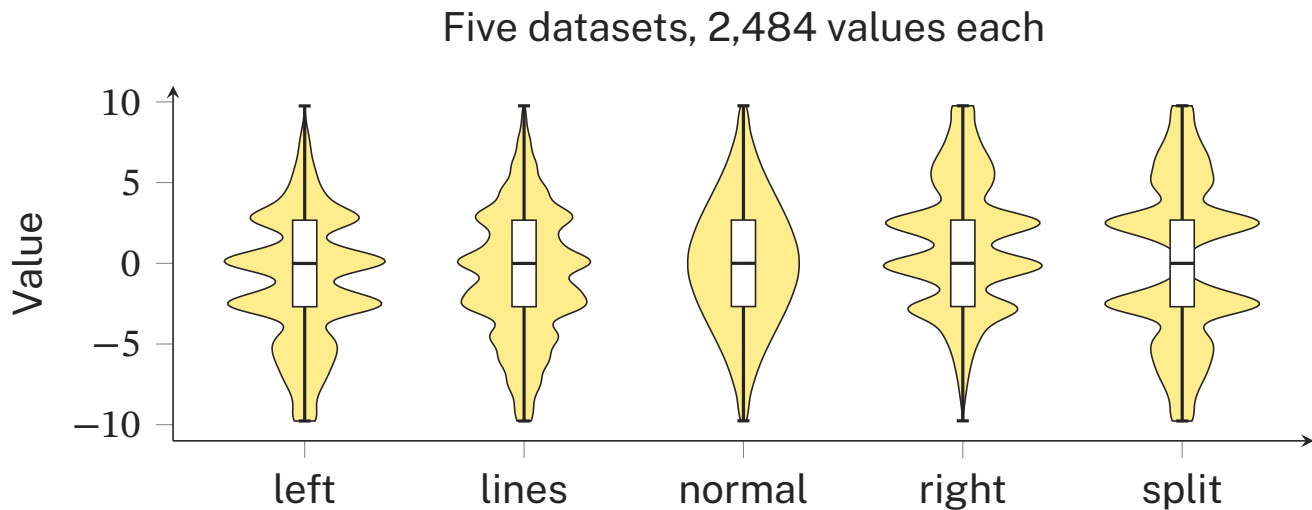


Figure 13. Five authentic teaching datasets from Matejka and Fitzmaurice, distributed through datasauRus. Each has 2,484 observations and nearly equal five-number summaries; the density shapes show what those summaries omit.

5 Cumulative distributions and quantile comparisons

5.1 The ECDF counts values at or below a threshold

For observations $x_1, \dots, x_n$, the empirical cumulative distribution function is $\hat{F}(t) = \#\{x_i \leq t\}/n$. It is right-continuous because the observation at a threshold is included. Each distinct value produces a jump; if m records are tied there, the jump is m/n . Below the smallest value the ECDF is zero; at and beyond the largest value it is one.

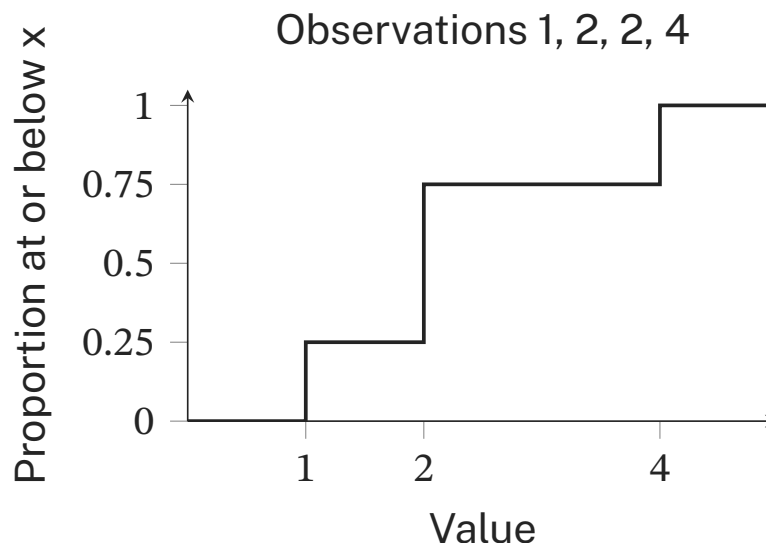


Figure 14. The observations 1, 2, 2, and 4 produce jumps of 0.25, 0.50, and 0.25. The combined jump at 2 accounts for the tie.

An ECDF requires no bins or smoothing bandwidth and works for both discrete and continuous measurements. It still depends on record selection, weights, and measurement validity. A missing value is not a numeric threshold, and a censored reporting limit is not an exact observed value. Report the number and treatment of eligible records before comparing ECDFs from course data.

For the 150 Iris petal widths, $\hat{F}(1.3) = 0.52$, meaning 78 recorded widths are at or below 1.3 cm. A flat section has no observations in its interior. A vertical comparison reads the difference in cumulative proportions at one measurement; a horizontal comparison reads the difference in sample quantiles at a chosen proportion.

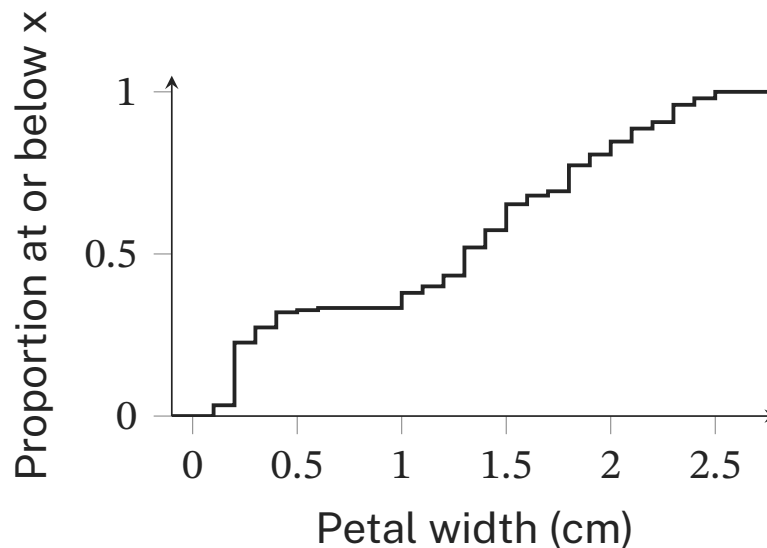


Figure 15. Petal-width ECDF with both its lower zero tail and upper one tail visible. Ties create jumps larger than $1/150$.

R

```
x <- d$Petal.Width
F_hat <- ecdf(x)
print(F_hat(1.3))
p_ecdf <- ggplot(d, aes(Petal.Width)) +
  stat_ecdf() +
  labs(x = "Petal width (cm)",
       y = "Proportion at or below x")
```

Python

```
x = d["Petal.Width"].to_numpy()
print(np.mean(x <= 1.3))
fig, ax = plt.subplots()
sns.ecdfplot(x=x, ax=ax)
ax.set(xlabel="Petal width (cm)",
       ylabel="Proportion at or below x")
```

The construction can be reproduced from distinct values and counts. `cumsum` accumulates counts and division by n converts them to proportions. The assertions check the arithmetic against the threshold calculation. To draw the complete result manually, use a right-continuous step and extend the zero and one tails; joining the distinct values with a sloped line would imply intermediate observed changes that are absent.

R

```
values <- sort(unique(x))
counts <- tabulate(match(x, values))
proportion <- cumsum(counts) / length(x)
print(data.frame(values, counts, proportion))
stopifnot(all.equal(proportion, F_hat(values)))
```

Python

```
values, counts = np.unique(
    x, return_counts=True)
proportion = np.cumsum(counts) / len(x)
print(pd.DataFrame({
    "values": values, "counts": counts,
    "proportion": proportion}))
direct = [(x <= v).mean() for v in values]
assert np.allclose(proportion, direct)
```

5.2 Compare groups on the same cumulative scale

All ECDFs end at one regardless of group size. In these data, setosa reaches one before versicolor begins. Versicolor's ECDF is at or above virginica's throughout the displayed range; they do not cross. This sample comparison places versicolor petal widths toward smaller values. It does not prove the same ordering for an entire population.

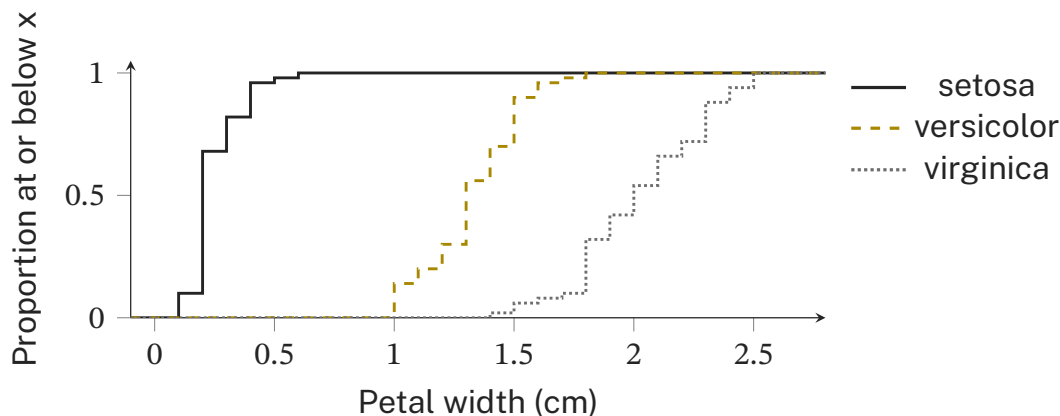


Figure 16. Species-specific petal-width ECDFs with a common scale. Distinct line patterns and an external legend preserve group identification where steps overlap.

R

```
p_group_ecdf <- ggplot(d,
  aes(Petal.Width, colour = Species,
    linetype = Species)) +
  stat_ecdf() +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Petal width (cm)",
    y = "Proportion at or below x")
```

Python

```
fig, ax = plt.subplots()
for s, c, line in zip(
    species, colours, styles):
    values = d.loc[d["Species"] == s,
        "Petal.Width"]
    sns.ecdfplot(values, color=c,
        linestyle=line, label=s,
        ax=ax)
ax.set(xlabel="Petal width (cm)",
    ylabel="Proportion at or below x")
ax.legend()
```

Other grouped ECDFs can cross, indicating that one ordering does not hold across all thresholds. Step curves can also coincide and hide one another. Use clear line patterns, selected direct labels, or separate aligned panels when overlap prevents the intended comparison.

5.3 Q-Q plots compare a sample with reference quantiles

A normal Q-Q plot puts ordered sample values on one axis and standard-normal quantiles on the other. A quantile is a threshold for a chosen cumulative proportion; the standard-normal reference is a bell-shaped distribution with mean zero and SD one. A distribution with a normal shape after shifting and

rescaling tends to produce an approximately straight pattern, with ordinary sampling variation around it. Systematic bends or tail departures suggest that a normal approximation may miss features of the sample.

Here the plotting positions are $p_i = (i - 0.5)/n$ for $i = 1, \dots, n$ and the horizontal values are $\Phi^{-1}(p_i)$, where Φ is the standard-normal cumulative distribution function. The reference line passes through the first and third sample quartiles. It supplies a geometric reference for describing the observed shape.

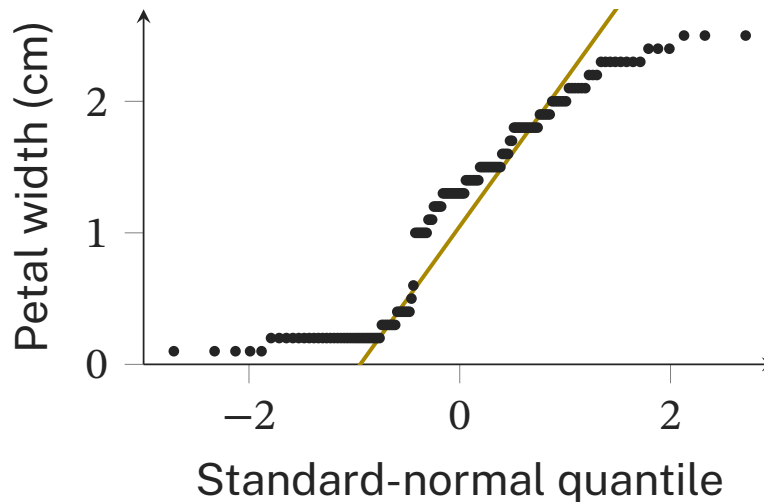


Figure 17. Iris petal-width normal Q-Q plot using the stated plotting positions and a quartile reference line with slope 1.11195 cm and intercept 1.05 cm.

R

```
n <- length(x)
q <- qnorm((seq_len(n) - 0.5) / n)
quartiles <- quantile(x, c(.25, .75))
slope <- diff(quartiles) /
  diff(qnorm(c(.25, .75)))
intercept <- quartiles[1] - slope * qnorm(.25)
qq_data <- data.frame(q = q, value = sort(x))
p_qq <- ggplot(qq_data, aes(q, value)) +
  geom_point() +
  geom_abline(intercept = intercept,
              slope = slope) +
  labs(x = "Standard-normal quantile",
       y = "Petal width (cm)")
```

Python

```
n = len(x)
prob = (np.arange(1, n + 1) - 0.5) / n
q = norm.ppf(prob)
quartiles = np.quantile(x, [.25, .75])
normal_iqr = np.diff(norm.ppf([.25, .75]))[0]
slope = np.diff(quartiles)[0] / normal_iqr
intercept = quartiles[0] - slope * norm.ppf(.25)
fig, ax = plt.subplots()
ax.scatter(q, np.sort(x))
ax.plot(q, intercept + slope * q)
ax.set(xlabel="Standard-normal quantile",
       ylabel="Petal width (cm)")
```

Petal width departs from this line. Its bounded, grouped measurements give context for the departure, but the Q-Q shape alone cannot identify a particular number of mixture components or its cause. R's `qqnorm`, SciPy's `probplot`, and statsmodels' `qqplot` are convenient alternatives; they can use different plotting positions and reference-line conventions. The explicit examples remove those differences for this comparison.

6 Visualizing proportions

6.1 A proportion needs a denominator

Suppose a fictional assembly has 157, 64, 73, and 26 seats for parties A, B, C, and D. The total is 320. Dividing each count by 320 gives the proportions 0.490625, 0.2, 0.228125, and 0.08125. These sum to one. Each graphic in this section is calculated from this same vector, with rounding used only in labels.

R

```
parts <- data.frame(
  Party = factor(LETTERS[1:4],
                levels = LETTERS[1:4]),
  Count = c(157, 64, 73, 26))
parts$Prop <- parts$Count / sum(parts$Count)
print(parts)
p_proportions <- ggplot(parts,
  aes(Party, Prop)) +
  geom_col() + labs(y = "Proportion")
```

Python

```
parts = pd.DataFrame({
  "Party": list("ABCD"),
  "Count": [157, 64, 73, 26]})
total = parts["Count"].sum()
parts["Prop"] = parts["Count"] / total
print(parts)
ax = parts.plot.bar(
  x="Party", y="Prop", legend=False)
ax.set_ylabel("Proportion")
```

A pie encodes these shares as angles or areas, while a bar chart uses lengths against a shared zero baseline. A and D are clearly different in either view; the smaller difference between B and C is easier to compare on aligned bars. This is a task-dependent design choice, not a universal ban on pies.

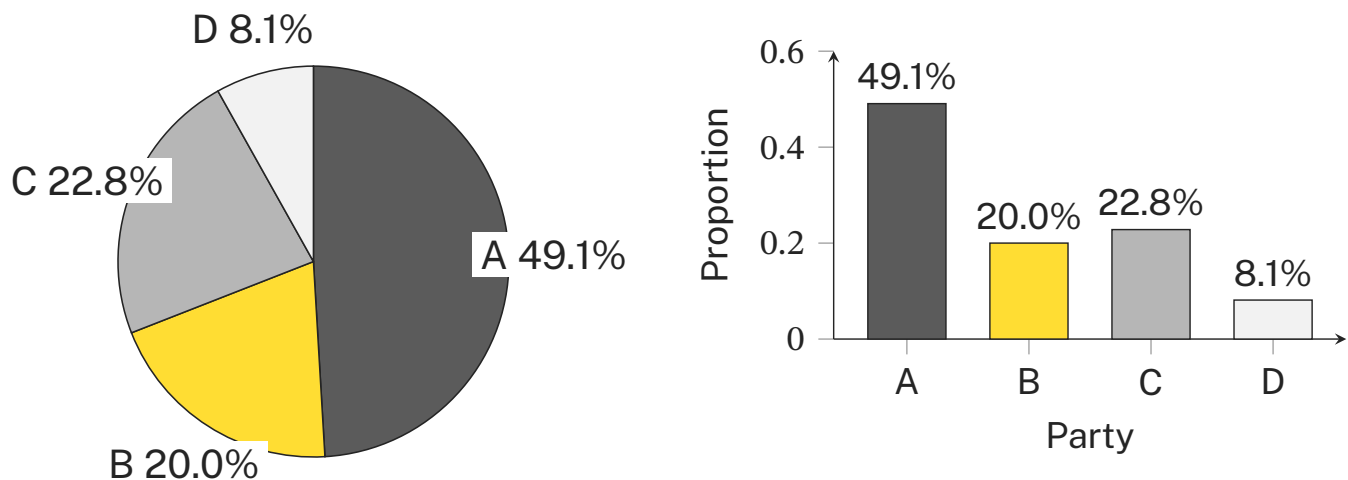


Figure 18. The same fictional seat shares as a pie and separate bars. Labels are rounded to one decimal percentage point; exact proportions determine the geometry.

6.2 Stacked bars emphasize composition

A single stack partitions a whole into adjacent segments. The four heights add to one, and the two interior segments lack a common baseline. Several normalized stacks can compare compositions between groups or dates, but cannot show their absolute total sizes. Include totals separately when a reader needs both. A stacked area chart is an optional extension over an ordered time axis; its filled areas require the same care with denominators and interior baselines.

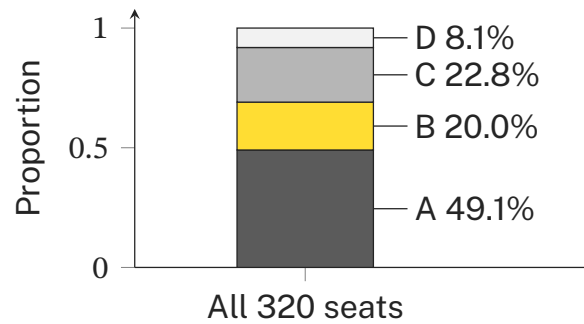


Figure 19. One stack of all four party shares. A is at the bottom, followed by B, C, and D, matching the explicit order in the examples.

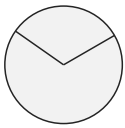
R

```
p_stacked <- ggplot(parts,
  aes(x = "All", y = Prop, fill = Party)) +
  geom_col(
    position = position_stack(reverse = TRUE)) +
  labs(x = NULL, y = "Proportion")
p_pie <- p_stacked + coord_polar(theta = "y") +
  theme_void()
```

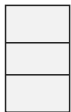
Python

```
one_row = parts.set_index("Party")[["Prop"]].T
stacked_ax = one_row.plot.bar(stacked=True)
stacked_ax.set(xlabel="", ylabel="Proportion")
fig, ax = plt.subplots()
ax.pie(parts["Count"], labels=parts["Party"],
      startangle=90, counterclock=False)
```

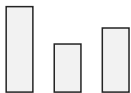
A pandas Series is one-dimensional, so transposing it does not turn four values into one stacked row. Selecting a one-column DataFrame and transposing that frame gives one row with four party columns. The R stack uses an explicit ordering option as well. Pie orientation can differ between plotting libraries without changing any share; identify categories in the legend or direct labels.



A pie shows parts of one whole.
Similar angles can be hard to compare.



A stack emphasizes composition.
Interior segments start at different heights.



Bars use a shared baseline.
Compare their lengths directly.

Figure 20. Pie angles, stacked segment lengths, and lengths on a common baseline support different comparison tasks. Direct labels can make exact shares easier to recover.

7 Visualizing associations

7.1 Start with a labelled scatter plot

A scatter plot places each observation at two numeric coordinates. Figure 21 uses petal length horizontally and sepal width vertically. Species is encoded by both colour and marker shape. Redundant encoding helps readers who cannot distinguish the colours and makes a greyscale copy more useful.

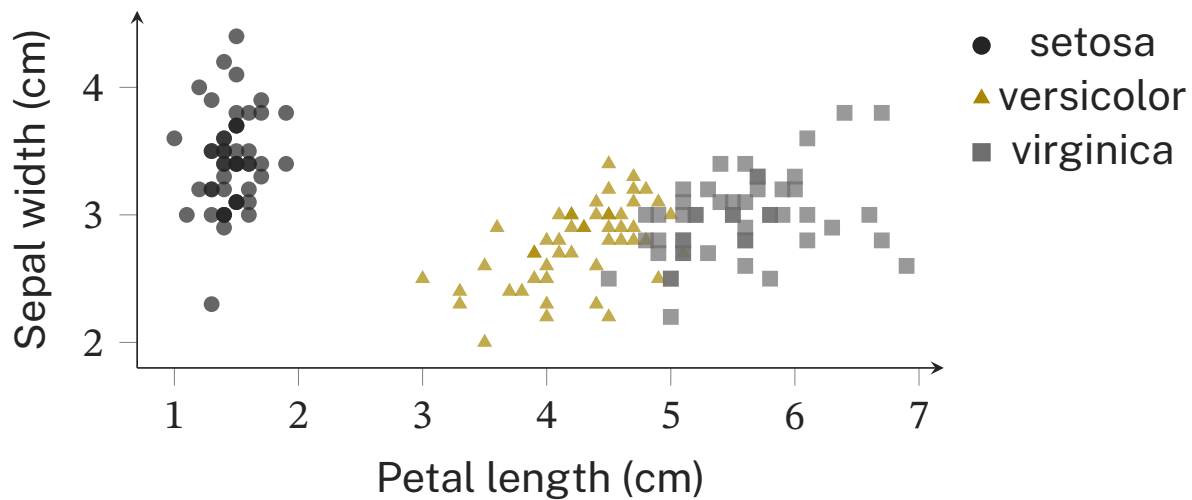


Figure 21. All 150 flowers with species shown by colour and shape. Both axes are measurements in centimetres, and the legend sits outside the data area.

R

```
p_scatter <- ggplot(d,
  aes(Petal.Length, Sepal.Width,
    colour = Species, shape = Species)) +
  geom_point(alpha = .7) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15)) +
  labs(x = "Petal length (cm)",
    y = "Sepal width (cm)")
```

Python

```
fig, ax = plt.subplots()
sns.scatterplot(
  data=d, x="Petal.Length", y="Sepal.Width",
  hue="Species", style="Species",
  hue_order=species, style_order=species,
  markers=markers, palette=colours,
  alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
  ylabel="Sepal width (cm)")
```

Transparency helps reveal coincident or dense points, although darkness from overplotting is not an exact density scale. Large samples may need smaller markers, aligned facets, or an explicit count-based display. Do not hide a relationship by selecting a narrow axis range without stating the restriction.

7.2 Use marker area only when a fourth variable helps

Marker area can carry another numeric measurement while position continues to show the original pair. The next plot maps area proportionally to sepal length on a scale beginning at zero. Within one marker shape, a value twice as large therefore receives twice the area, not twice the radius. Circles, triangles, and squares have different area constants for the same size parameter, so compare marker sizes within one species. The circular size key illustrates the circle mapping; it does not calibrate the other shapes. R and Python also use different physical marker-size units. Exact cross-species area comparisons would require a common shape or a separately calibrated key for each shape.

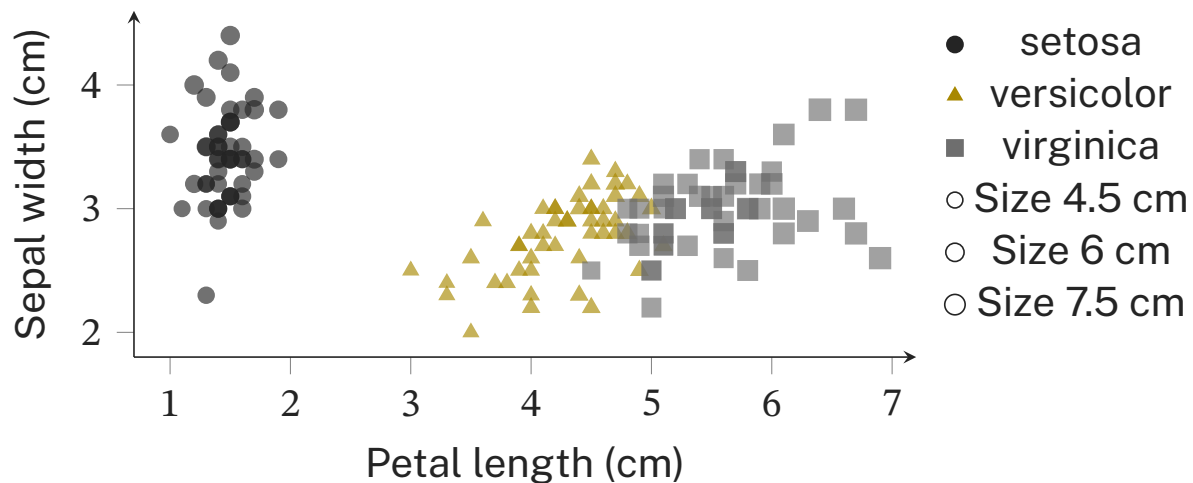


Figure 22. Petal length, sepal width, species, and sepal length in one scatter plot. The circular size key is in centimetres; proportional area comparisons apply within one marker shape.

R

```
p_size <- p_scatter + aes(size = Sepal.Length) +
  scale_size_area(
    max_size = 5,
    limits = c(0, max(d$Sepal.Length)),
    breaks = c(4.5, 6, 7.5),
    name = "Sepal length (cm)")
```

Python

```
fig, ax = plt.subplots()
maximum = d["Sepal.Length"].max()
sns.scatterplot(
    data=d, x="Petal.Length", y="Sepal.Width",
    hue="Species", style="Species",
    hue_order=species, style_order=species,
    markers=markers, palette=colours,
    size="Sepal.Length", size_norm=(0, maximum),
    sizes=(0, 120), alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
      ylabel="Sepal width (cm)")
```

The extra variable costs attention. If exact comparisons of sepal length are central to the question, use a separate aligned plot or facets rather than requiring readers to judge small area differences.

7.3 Explain pooled and within-group correlation separately

Pearson's correlation coefficient r summarizes linear association. It ranges from -1 to $+1$ and has no measurement unit. For nonconstant paired variables,

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}.$$

A value near zero does not exclude a nonlinear relationship. A large absolute value does not establish causation and can be sensitive to unusual observations. Always return to the scatter plot and the data-collection context.

For petal length and sepal width, the pooled coefficient across all 150 flowers is -0.428 . Within the 50-flower groups it is $+0.178$ for setosa, $+0.561$ for versicolor, and $+0.401$ for virginica. Thus between-species differences and within-species variation produce different relationships. Pooling is not a neutral replacement for a grouped question.

R

```
print(cor(d$Petal.Length, d$Sepal.Width))
by_species <- by(d, d$Species, function(g)
  cor(g$Petal.Length, g$Sepal.Width))
print(by_species)
correlations <- cor(d[numeric_names])
print(round(correlations, 3))
```

Python

```
print(d["Petal.Length"].corr(d["Sepal.Width"]))
by_species = {}
for s, g in d.groupby("Species", observed=True):
    by_species[s] = g["Petal.Length"].corr(
        g["Sepal.Width"])
print(by_species)
correlations = d[numeric_names].corr()
print(correlations.round(3))
```

For another dataset, identify the eligible paired records and report their number. Pairwise handling of missing measurements can give different record sets to different coefficients. A correlation matrix should not conceal those denominator changes; the complete Iris example has no missing measurements.

8 Several variables in one overview

8.1 Scatter-plot matrices repeat pairs systematically

A four-variable matrix contains $4 \times 4 = 16$ panels. The four diagonal panels show one-variable distributions. There are $4(4 - 1)/2 = 6$ distinct variable pairs; each appears twice, once on each side of the diagonal with its axes exchanged. The number of unique panels including diagonals is $4(4 + 1)/2 = 10$, but the full matrix still draws 16 panels. Select the same four numeric variables in both languages; species is a grouping variable rather than a fifth numeric axis.

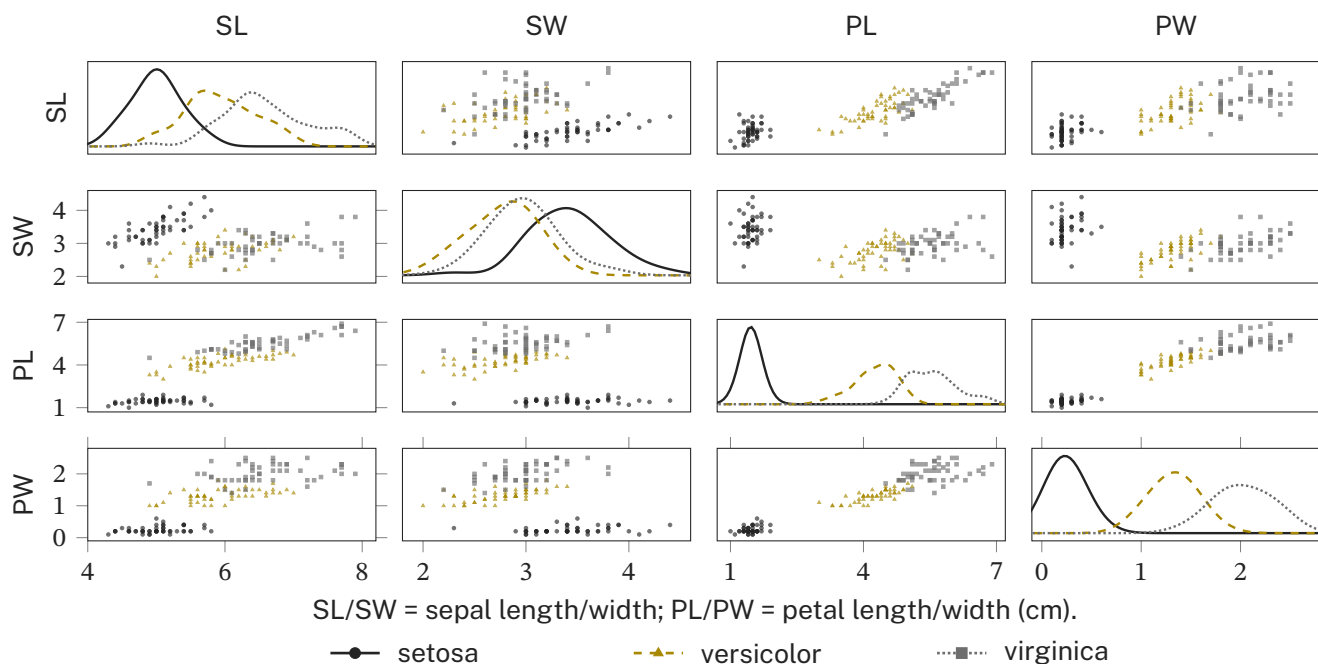


Figure 23. Four-variable Iris matrix. In off-diagonal panels, row labels identify the vertical variable and column labels the horizontal variable. Diagonal vertical axes show within-species density instead. Outer ticks give measurement scales in centimetres; colour, shape, and line style identify species.

R

```
p_matrix <- ggpairs(d, columns = 1:4,
  mapping = aes(colour = Species,
    shape = Species),
  upper = list(continuous = "points"),
  lower = list(continuous = "points"),
  diag = list(continuous = wrap(
    "densityDiag", bw = h_group))) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15))
```

Python

```
matrix = sns.pairplot(
  d, vars=numeric_names, hue="Species",
  palette=colours, markers=markers,
  diag_kind="kde",
  diag_kws={"bw_method": bandwidth,
    "common_norm": False})
```

GGally's `ggpairs` can put summaries such as correlations in its upper triangle [6]. The example requests points in both triangles to match the displayed matrix. A matrix is an overview, so select and enlarge a pair before making a detailed numerical claim. Upper and lower panels should not receive duplicate point layers from a later annotation call.

8.2 Correlograms compress linear associations

A correlogram maps entries of the correlation matrix to colour. The diagonal is one because a non-constant variable is perfectly correlated with itself; the matrix is symmetric because $r(x, y) = r(y, x)$. A diverging colour scale with fixed endpoints -1 and $+1$ distinguishes signs and keeps separate matrices comparable. Colour alone is a coarse summary, so exact values can be printed inside the cells or supplied in a table.

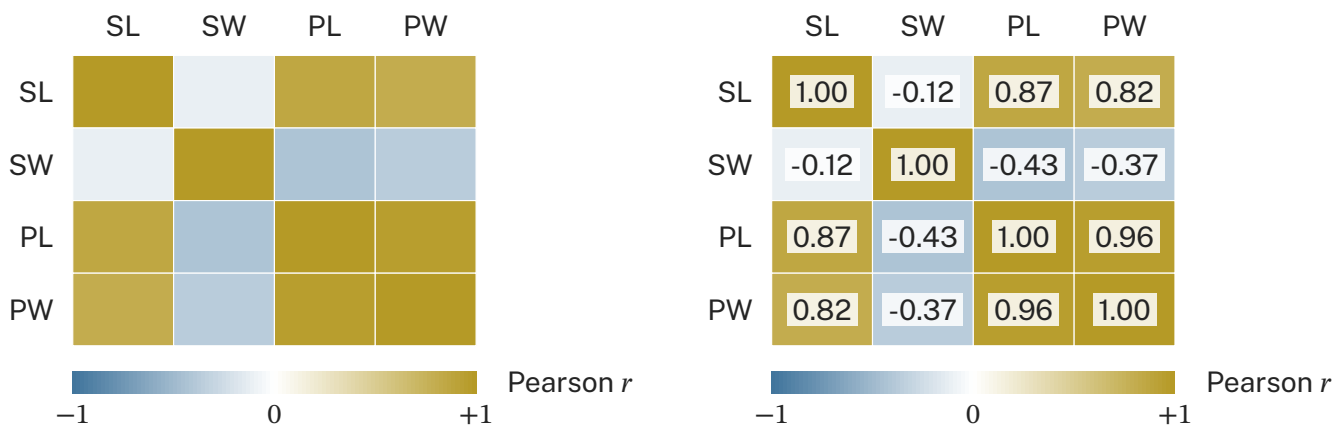


Figure 24. One computed correlation matrix shown as colour and as colour with values. SL, SW, PL, and PW mean sepal length, sepal width, petal length, and petal width. Colour uses the same bounded numerical scale in both panels.

Table 3. The six unique pooled Iris correlations, all based on 150 paired observations.

Variable pair	Pearson r
Sepal length / Sepal width	-0.118
Sepal length / Petal length	0.872
Sepal length / Petal width	0.818
Sepal width / Petal length	-0.428
Sepal width / Petal width	-0.366
Petal length / Petal width	0.963

R

```
cells <- as.data.frame(as.table(correlations))
p_correlogram <- ggplot(cells,
  aes(Var1, Var2, fill = Freq)) + geom_tile() +
  geom_text(aes(label = sprintf("%.2f", Freq))) +
  scale_fill_gradient2(
    low = "#506C85", mid = "white",
    high = "#A88800", limits = c(-1, 1)) +
  coord_equal() +
  labs(x = NULL, y = NULL, fill = "r")
```

Python

```
fig, ax = plt.subplots()
palette = sns.diverging_palette(
  240, 60, as_cmap=True)
sns.heatmap(
  correlations, annot=True, fmt=".2f",
  vmin=-1, vmax=1, center=0, square=True,
  cmap=palette, ax=ax)
```

Petal length and width have $r = 0.963$, so a two-decimal cell reads 0.96. Rounding to 1.0 would suggest perfect correlation when residual variation remains. A correlogram hides nonlinearity, individual outliers, and group-specific patterns. Use it to select relationships to inspect, not to replace scatter plots.

8.3 Parallel coordinates show profiles across columns

Parallel coordinates place variables on adjacent vertical axes and connect the values from each row. Iris measurements are standardized as $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$ using the sample SD with divisor $n - 1$. The vertical axis is therefore unitless, and zero denotes the column's mean. One flower contributes one line across all four measurements.

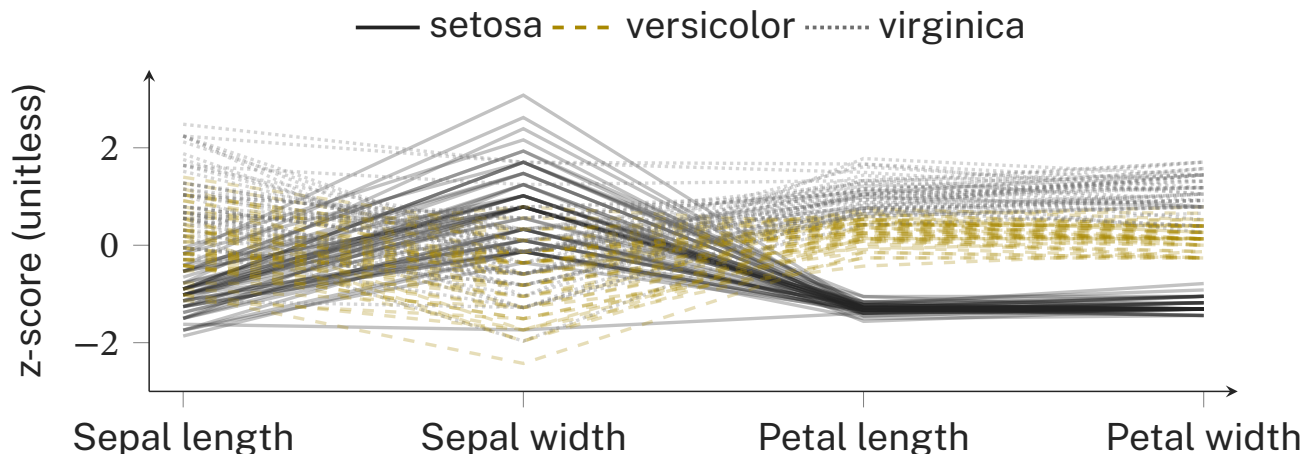


Figure 25. All 150 standardized flower profiles. Colour identifies species; values are sample-standard-deviation units from each column's mean. The four-axis order is explicit.

R

```
z <- scale(d[numeric_names])
n <- nrow(d)
variable <- factor(numeric_names,
                    levels = numeric_names)
long <- data.frame(Row = rep(seq_len(n), 4),
                  Variable = rep(variable, each = n),
                  Z = as.vector(z), Species = rep(d$Species, 4))
p_parallel <- ggplot(long, aes(Variable, Z,
                              group = Row, colour = Species,
                              linetype = Species)) + geom_line(alpha = .35) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = NULL, y = "z-score (unitless)")
```

Python

```
z = d[numeric_names]
z = (z - z.mean()) / z.std(ddof=1)
fig, ax = plt.subplots()
for s, c, line in zip(species, colours, styles):
    rows = z.loc[d["Species"] == s].to_numpy()
    ax.plot(range(4), rows.T, color=c,
            linestyle=line, alpha=.35)
    ax.plot([], [], color=c, linestyle=line,
            label=s)
ax.set_xticks(range(4), numeric_names)
ax.set_ylabel("z-score (unitless)")
ax.legend()
```

R's `scale` and pandas' `std(ddof=1)` agree on this sample-SD convention. A population-SD scaling would produce slightly different coordinates. A constant column cannot be divided by its zero SD and needs separate treatment. Standardization does not remove outliers or imply that each variable is equally useful. Reordering axes changes which crossings are visible; dense crossing patterns should not be read as causal paths between variables.

9 Choose, critique, and explain a chart

A chart chooser such as Abela's offers candidate displays [1]. Perception-focused guidance explains why aligned positions and clear grouping can make a comparison easier [2]. Use those ideas to ask four questions before publishing a figure.

1. What comparison should the audience make?
2. Do the variables, units, grouping, and denominator support that comparison?
3. Could binning, smoothing, ordering, or axis limits change the conclusion?
4. Can a reader identify the groups and reproduce the displayed values?

There is no precision ladder from histogram to density to ECDF to Q-Q plot. These charts answer different questions. Likewise, grouped boxes and violins trade a compact summary against estimated shape; neither automatically improves on the other. For proportions, aligned bars often support close comparisons. For associations, inspect the points and groups before reducing them to a number.

9.1 Ungraded practice and the Lab 03 connection

The small worked examples are ungraded practice. The Lab 03 handout defines the assessed task and deliverables. Work in R or Python and adapt the taught plotting operations to the cleaned variables in that handout.

1. Draw two histograms of one variable using documented edges.
2. Add an ECDF or a box plot and explain what each view reveals.
3. Inspect a numeric pair, then distinguish meaningful groups.
4. Write a caption naming units, eligible records, the observed pattern, and one limitation. Keep the conclusion descriptive.

As optional ungraded practice, choose one public chart from a news article, a research paper, or a government statistical agency. Identify its question, data source, units, and denominator. Explain

one design choice that supports or weakens the claim, then sketch an alternative without inventing unavailable data. Credit the chart and distinguish a design critique from a claim that its data are wrong.

Optional further reading

- ggplot2 histograms and NumPy histograms explain edges, counts, and endpoint rules.
- ggplot2 box plots and seaborn violin plots document quartiles, whiskers, bandwidth, and scaling.
- Claus Wilke, Fundamentals of Data Visualization develops chart choice, distributions, proportions, and associations.

Next week

Class 07 develops manipulation and visualization in R or Python. Class 08 connects code, figures, and interpretation in R Markdown or Python notebooks. The same data-preparation and plotting decisions become part of a reproducible report. Predictive modelling is introduced later in the syllabus sequence.

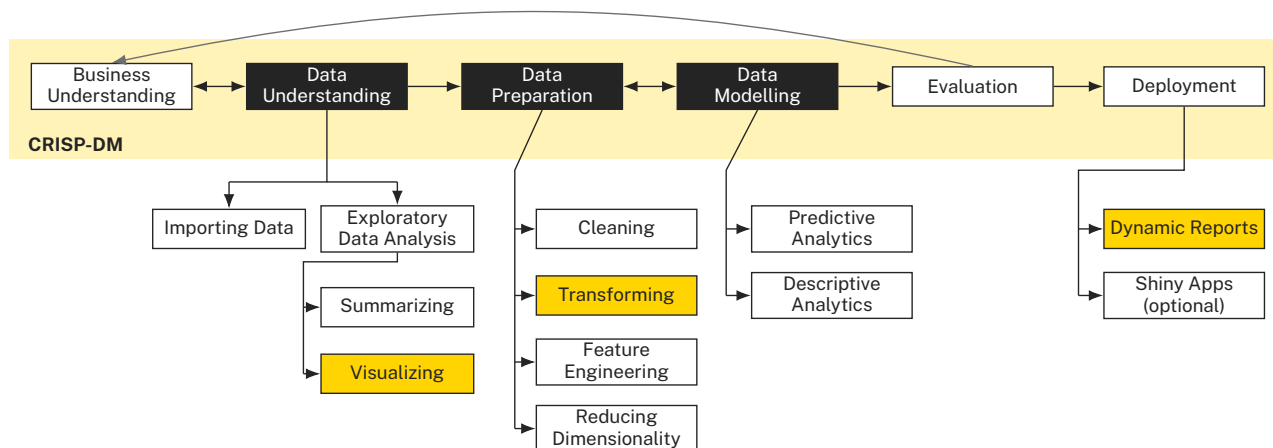


Figure 26. Week 4 connects transforming, visualizing, and dynamic reports within the same course workflow.

Integrated lab | Lab 03 | Visualizing changing Canadian environments

Build a coherent visual argument about change across time, place, sector, or category.

- **Carry forward.** Week 01 summaries and the Week 02 cleaning pipeline.
- **Start in class.** Choose one verified dataset, work through the selection checklist, and set up the R or Python analysis.
- **Assessment source.** week-03/Assessment/week-03-lab.pdf

References

- [1] A. Abela. *Chart Suggestions: A Thought-Starter*. Chart chooser. 2009. URL: <https://extremepresentation.com/wp-content/uploads/choosing-a-good-chart-09-1.pdf>.
- [2] S. Few. “Data visualization for human perception”. In: *The Encyclopedia of Human-Computer Interaction*. 2nd ed. Interaction Design Foundation.
- [3] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.
- [4] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 2017, pp. 1290–1294.
- [5] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- [6] B. Schloerke, D. Cook, J. Larmarange, F. Briatte, M. Marbach, E. Thoen, A. Elberg, and J. Crowley. *GGally: Extension to ggplot2*. Documentation for ggpairs, ggcorr and ggparcoord. CRAN.
- [7] H. Wickham, D. Navarro, and T. L. Pedersen. *ggplot2: Elegant Graphics for Data Analysis*. 3rd ed. Online edition, work in progress. Springer.